

Real Data Simulations

Dohyun (Andy) Ju

2026-04-28

Load in the data

```
library(Seurat)
library(countsplit)
library(Matrix)
library(ggplot2)
library(patchwork)
library(mclust)
library(countsplit.tutorials)
library(CADET)
library(coop)
library(tidyverse)
library(dplyr)
library(purrr)
```

```
# Retrieve Zheng dataset
library(DuoClustering2018)
sce <- sce_full_Zhengmix4eq()
```

```
## see ?DuoClustering2018 and browseVignettes('DuoClustering2018') for documentation
## loading from cache
## require("SingleCellExperiment")
```

Create raw count matrices for 1-cluster and 2-cluster cases

The 1-cluster case only contains B cells and the 2-cluster case contains B cells and CD14 monocytes.

```
# This is the genes (rows) x cells (columns) matrix
X.sce <- counts(sce)
dim(X.sce)
```

```
## [1] 15568 3994
```

```
# These are the true cell types
cm <- as.data.frame(colData(sce))
true_cell_types <- cm$phenoid
table(true_cell_types)
```

```
## true_cell_types
##      b.cells  cd14.monocytes naive.cytotoxic  regulatory.t
##          999          1000           998           997
```

```
# Subsetting data for power and type 1 error comparisons
# Subset data for only b cells (1 true cluster)
```

```

b.cells.names <- rownames(subset(cm, phenoid == "b.cells"))
X.b.cells <- X.sce[,b.cells.names]
dim(X.b.cells)

## [1] 15568    999

# Subset data for b cells and cd14 monocytes (2 true clusters)
b.cd14.names <- rownames(subset(cm, (phenoid == "b.cells" | phenoid == "cd14.monocytes")))
X.b.cd14 <- X.sce[,b.cd14.names]
dim(X.b.cd14)

## [1] 15568    1999

```

1. The null case (B cells only)

Preprocessing

```

b.cells.seurat <- CreateSeuratObject(counts = X.b.cells, min.cells = 3, min.features = 200)

## Warning: Data is of class matrix. Coercing to dgCMatix.

b.cells.seurat[["percent.mt"]] <- PercentageFeatureSet(b.cells.seurat, pattern = "^MT-")
b.cells.seurat <- subset(b.cells.seurat, subset = nFeature_RNA > 200 & nFeature_RNA < 2500 & percent.mt

```

Normalizing and scaling data

```

b.cells.seurat <- NormalizeData(b.cells.seurat)
b.cells.seurat <- FindVariableFeatures(b.cells.seurat, selection.method = "vst", nfeatures = 2000)
b.cells.genes <- rownames(b.cells.seurat)
b.cells.seurat <- ScaleData(b.cells.seurat, features = b.cells.genes)
b.cells.seurat <- RunPCA(b.cells.seurat, features = VariableFeatures(object = b.cells.seurat))

```

Countsplit

```
source("run_countsplit.R")
```

We must run countsplit to get the top 500 most variable genes (since we want to ensure all genes contained within the top 500 list are in the train/test sets created by countsplit). It is likely that these smaller datasets do not contain all the same genes as the complete dataset, which is why we must use the training set to create the top 500 list.

```

# Countsplit
for(i in 1:10) {
  run_countsplit(X.b.cells, seed = i, null_case = TRUE)
}

# Get the top 500 genes for each random seed
top.500.files.bcells <- list.files(pattern = "top.500.b.cells.*\\.Rds$")

for (f in top.500.files.bcells) {
  obj_name <- gsub("\\.Rds", "", f)

  # Read the file and assign it to that name in the Global Environment
  assign(obj_name, readRDS(f), envir = .GlobalEnv)
}

```

```
top500_files_list.bcells <- paste0("top.500.b.cells.", 1:10) # for correct order of files
```

Naive method

```
source("run_naive.R")

# Get PC scores (scores for each cell); based on previous runs, 10 seems to be sufficient
b.cells.pca_scores <- Embeddings(b.cells.seurat, reduction = "pca")[, 1:10]

# Normalize and scale raw counts
X.b.cells.norm <- t(X.b.cells / colSums(X.b.cells)) * 10000 # normalize to size factors and scale
X.b.cells.t <- log1p(X.b.cells.norm) # log normalize

# Naive method
for(i in 1:10) {
  top.500_file <- get(top500_files_list.bcells[i])
  run_naive(X.b.cells.t, seed = i, top.500 = top.500_file, pca_scores = b.cells.pca_scores,
            null_case = TRUE)
}
```

CADET (null, zero and non-zero diagonals)

```
source("run_cadet_null.R")

for(i in 1:10) {
  top.500_file <- get(top500_files_list.bcells[i])
  run_cadet_null(X.b.cells.t, seed = i, top.500 = top.500_file, null_case = TRUE)
}
```

CADET (demean, zero and non-zero diagonals)

```
source("run_cadet_demean.R")

for(i in 1:10) {
  top.500_file <- get(top500_files_list.bcells[i])
  run_cadet_demean(X.b.cells.t, seed = i, top.500 = top.500_file, null_case = TRUE)
}
```

2. The 2-cluster case (B cells and CD14 monocytes)

Preprocessing

```
b.cd14.seurat <- CreateSeuratObject(counts = X.b.cd14, min.cells = 3, min.features = 200)

## Warning: Data is of class matrix. Coercing to dgCMatix.
b.cd14.seurat[["percent.mt"]] <- PercentageFeatureSet(b.cd14.seurat, pattern = "^MT-")
b.cd14.seurat <- subset(b.cd14.seurat, subset = nFeature_RNA > 200 & nFeature_RNA < 2500 & percent.mt <
```

Normalizing and scaling data

```
b.cd14.seurat <- NormalizeData(b.cd14.seurat)
b.cd14.seurat <- FindVariableFeatures(b.cd14.seurat, selection.method = "vst", nfeatures = 2000)
```

```
b.cd14.genes <- rownames(b.cd14.seurat)
b.cd14.seurat <- ScaleData(b.cd14.seurat, features = b.cd14.genes)
b.cd14.seurat <- RunPCA(b.cd14.seurat, features = VariableFeatures(object = b.cd14.seurat))
```

Countsplit

```
# Countsplit
for(i in 1:10) {
  run_countsplit(X.b.cd14, seed = i, null_case = FALSE)
}

# Get the top 500 genes for each random seed
top.500.files.bcd14 <- list.files(pattern = "top.500.b.cd14.*\\.Rds$")

for (f in top.500.files.bcd14) {
  obj_name <- gsub("\\.Rds", "", f)

  # Read the file and assign it to that name in the Global Environment
  assign(obj_name, readRDS(f), envir = .GlobalEnv)
}

top500_files_list.bcd14 <- paste0("top.500.b.cd14.", 1:10) # for correct order of files
```

Naive method

```
# Get PC scores (scores for each cell); based on previous runs, 10 seems to be sufficient
b.cd14.pca_scores <- Embeddings(b.cd14.seurat, reduction = "pca")[, 1:10]

# Normalize and scale raw counts
X.b.cd14.norm <- t(X.b.cd14 / colSums(X.b.cd14)) * 10000 # normalize to size factors and scale
X.b.cd14.t <- log1p(X.b.cd14.norm) # log normalize

# Naive method
for(i in 1:10) {
  top.500_file <- get(top500_files_list.bcd14[i])
  run_naive(X.b.cd14.t, seed = i, top.500 = top.500_file, pca_scores = b.cd14.pca_scores,
    null_case = FALSE)
}
```

CADET (null, zero and non-zero diagonals)

```
for(i in 1:10) {
  top.500_file <- get(top500_files_list.bcd14[i])
  run_cadet_null(X.b.cd14.t, seed = i, top.500 = top.500_file, null_case = FALSE)
}
```

CADET (demean, zero and non-zero diagonals)

```
for(i in 1:10) {
  top.500_file <- get(top500_files_list.bcd14[i])
  run_cadet_demean(X.b.cd14.t, seed = i, top.500 = top.500_file, null_case = FALSE)
}
```

3. Analysis: 1 cluster (null) case

Loading in the data

```
# Loading naive p-values
path_naive_bcells <- "~/Documents/scRNAseq Neufeld Research/Winter Spring 2026/Real_Data_Simulations/pvals/naive_bcells"
naive_files_bcells <- list.files(path = path_naive_bcells, pattern = "\\Rds$", full.names = TRUE)
clean_names_naive <- tools::file_path_sans_ext(basename(naive_files_bcells))
naive_pvals_bcells <- setNames(lapply(naive_files_bcells, readRDS), clean_names_naive)

# reorder because R orders 10 in front of 2
naive_pvals_bcells <- naive_pvals_bcells[c(1,3,4,5,6,7,8,9,10,2)]

# Loading countsplit p-values
path_countsplit_bcells <- "~/Documents/scRNAseq Neufeld Research/Winter Spring 2026/Real_Data_Simulations/pvals/countsplit_bcells"
countsplit_files_bcells <- list.files(path = path_countsplit_bcells, pattern = "\\Rds$", full.names = TRUE)
clean_names_countsplit_bcells <- tools::file_path_sans_ext(basename(countsplit_files_bcells))
countsplit_pvals_bcells <- setNames(lapply(countsplit_files_bcells, readRDS), clean_names_countsplit_bcells)

# reorder because R orders 10 in front of 2
countsplit_pvals_bcells <- countsplit_pvals_bcells[c(1,3,4,5,6,7,8,9,10,2)]

# Loading CADET null (nonzero off diagonals) p-values
path_cadet_null_bcells <- "~/Documents/scRNAseq Neufeld Research/Winter Spring 2026/Real_Data_Simulations/pvals/cadet_null_bcells"
cadet_null_files_bcells <- list.files(path = path_cadet_null_bcells, pattern = "\\Rds$", full.names = TRUE)
clean_names_cadet_null_bcells <- tools::file_path_sans_ext(basename(cadet_null_files_bcells))
cadet_null_pvals_bcells <- setNames(lapply(cadet_null_files_bcells, readRDS), clean_names_cadet_null_bcells)

# reorder because R orders 10 in front of 2
cadet_null_pvals_bcells <- cadet_null_pvals_bcells[c(1,3,4,5,6,7,8,9,10,2)]

# Loading CADET null indep p-values
path_cadet_indep_null_bcells <- "~/Documents/scRNAseq Neufeld Research/Winter Spring 2026/Real_Data_Simulations/pvals/cadet_indep_null_bcells"
cadet_indep_null_files_bcells <- list.files(path = path_cadet_indep_null_bcells, pattern = "\\Rds$", full.names = TRUE)
clean_names_cadet_indep_null_bcells <- tools::file_path_sans_ext(basename(cadet_indep_null_files_bcells))
cadet_indep_null_pvals_bcells <- setNames(lapply(cadet_indep_null_files_bcells, readRDS), clean_names_cadet_indep_null_bcells)

# reorder because R orders 10 in front of 2
cadet_indep_null_pvals_bcells <- cadet_indep_null_pvals_bcells[c(1,3,4,5,6,7,8,9,10,2)]

# Loading CADET demean (nonzero off diagonals) p-values
path_cadet_demean_bcells <- "~/Documents/scRNAseq Neufeld Research/Winter Spring 2026/Real_Data_Simulations/pvals/cadet_demean_bcells"
cadet_demean_files_bcells <- list.files(path = path_cadet_demean_bcells, pattern = "\\Rds$", full.names = TRUE)
clean_names_cadet_demean_bcells <- tools::file_path_sans_ext(basename(cadet_demean_files_bcells))
cadet_demean_pvals_bcells <- setNames(lapply(cadet_demean_files_bcells, readRDS), clean_names_cadet_demean_bcells)

# reorder because R orders 10 in front of 2
cadet_demean_pvals_bcells <- cadet_demean_pvals_bcells[c(1,3,4,5,6,7,8,9,10,2)]

# Loading CADET demean indep p-values
path_cadet_indep_demean_bcells <- "~/Documents/scRNAseq Neufeld Research/Winter Spring 2026/Real_Data_Simulations/pvals/cadet_indep_demean_bcells"
cadet_indep_demean_files_bcells <- list.files(path = path_cadet_indep_demean_bcells, pattern = "\\Rds$", full.names = TRUE)
clean_names_cadet_indep_demean_bcells <- tools::file_path_sans_ext(basename(cadet_indep_demean_files_bcells))
cadet_indep_demean_pvals_bcells <- setNames(lapply(cadet_indep_demean_files_bcells, readRDS), clean_names_cadet_indep_demean_bcells)

# reorder because R orders 10 in front of 2
cadet_indep_demean_pvals_bcells <- cadet_indep_demean_pvals_bcells[c(1,3,4,5,6,7,8,9,10,2)]
```

```
cadet_indep_demean_pvals_bcells <- cadet_indep_demean_pvals_bcells[c(1,3,4,5,6,7,8,9,10,2)]
```

P-value analysis for uniformity

```
naivecol <- "#E763F3"
countsplitcol <- "#00BF7D"
cadet_nullcol <- "#E7861B"
cadet_indep_nullcol <- "darkblue"
cadet_demeancol <- "cyan"
cadet_indep_demeancol <- "yellow"

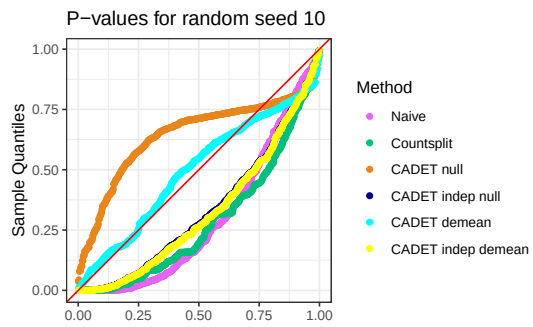
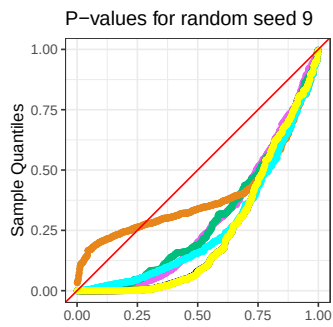
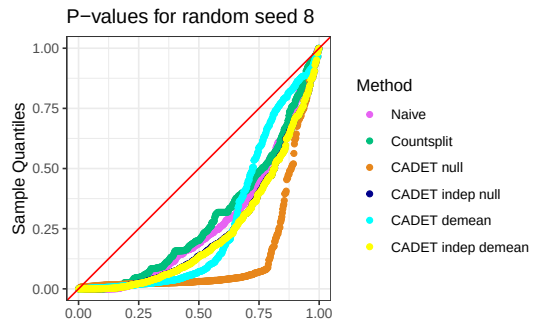
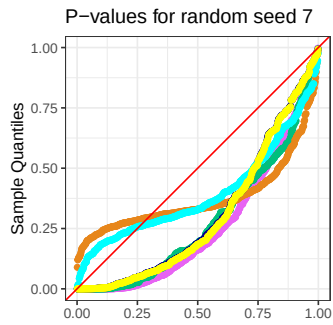
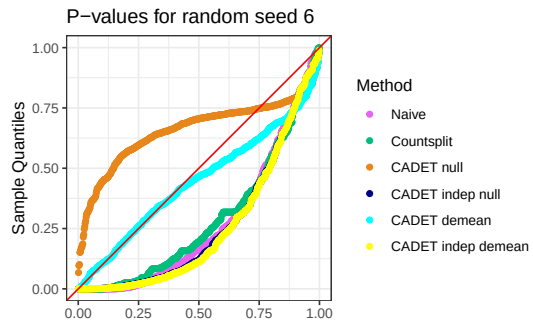
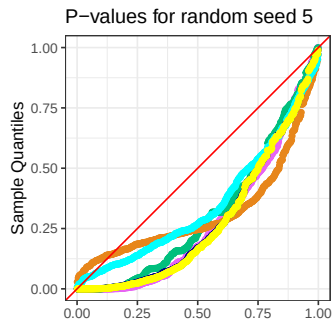
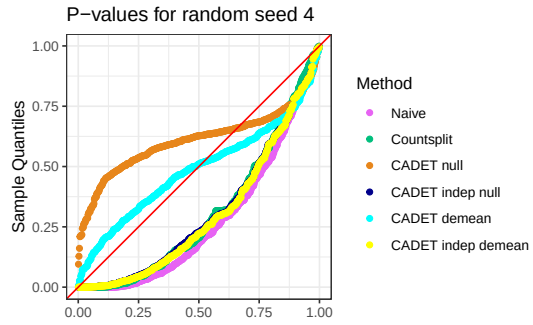
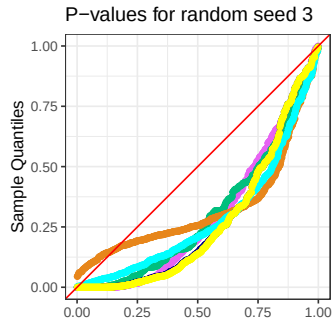
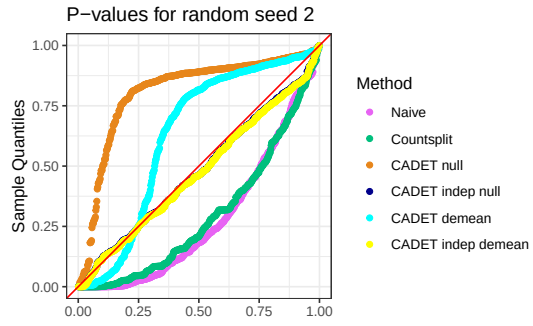
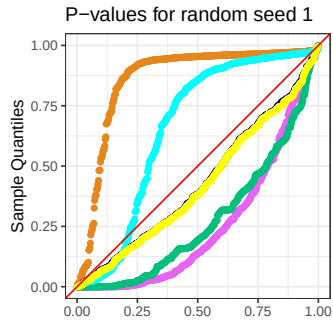
qunif_bcells_plots <- list()

for (i in 1:10) {
  pvals_list <- list(naive_pvals_bcells[[i]], countsplit_pvals_bcells[[i]],
                    cadet_null_pvals_bcells[[i]], cadet_indep_null_pvals_bcells[[i]],
                    cadet_demean_pvals_bcells[[i]], cadet_indep_demean_pvals_bcells[[i]])
  pvals_list <- pvals_list %>% purrr::reduce(full_join, by = "Gene")
  colnames(pvals_list) <- c("Gene", "naive", "countsplit", "cadet_null",
                           "cadet_indep_null", "cadet_demean", "cadet_indep_demean")

  bcells_unifplot <- ggplot(data=pvals_list)+
    geom_qq(aes(sample=naive, col="Naive"), distribution = stats::qunif)+
    geom_qq(aes(sample=countsplit, col="Countsplit"), distribution = stats::qunif)+
    geom_qq(aes(sample=cadet_null, col="CADET null"), distribution = stats::qunif)+
    geom_qq(aes(sample=cadet_indep_null, col="CADET indep null"), distribution=stats::qunif)+
    geom_qq(aes(sample=cadet_demean, col="CADET demean"), distribution = stats::qunif)+
    geom_qq(aes(sample=cadet_indep_demean, col="CADET indep demean"),
            distribution = stats::qunif)+
    ggtitle("All")+
    coord_fixed()+
    theme_bw()+
    geom_abline(slope=1, intercept=0, col="red")+
    xlab("Unif(0,1) Quantiles")+
    ylab("Sample Quantiles")+
    scale_color_manual(
      limits = c("Naive", "Countsplit", "CADET null", "CADET indep null",
                 "CADET demean", "CADET indep demean"),
      values=c(naivecol, countsplitcol, cadet_nullcol, cadet_indep_nullcol, cadet_demeancol,
               cadet_indep_demeancol))+
    ggtitle(paste("P-values for random seed", i)) +
    labs(col="Method")+xlab(" ")

  qunif_bcells_plots[[i]] <- bcells_unifplot
}

library(patchwork)
wrap_plots(qunif_bcells_plots, ncol = 2)
```



Results for null (1 cluster) case: - CADET without independence assumption = highly random seed dependent
 - Even with independence: CADET still random seed dependence - Demean vs. no demean does not look very different (which makes sense because demean should not make much of a difference for the 1 cluster case)
 - Countsplit looks consistent across random seeds, but doesn't seem to improve much on naive

4. Analysis: 2 cluster case

```
# Loading naive p-values
path_naive_bcd14 <- "~/Documents/scRNAseq Neufeld Research/Winter Spring 2026/Real_Data_Simulations/pvals_naive_bcd14"
naive_files_bcd14 <- list.files(path = path_naive_bcd14, pattern = "\\Rds$", full.names = TRUE)
clean_names_naive_bcd14 <- tools::file_path_sans_ext(basename(naive_files_bcd14))
naive_pvals_bcd14 <- setNames(lapply(naive_files_bcd14, readRDS), clean_names_naive_bcd14)

# reorder because R orders 10 in front of 2
naive_pvals_bcd14 <- naive_pvals_bcd14[c(1,3,4,5,6,7,8,9,10,2)]

# Loading countsplit p-values
path_countsplit_bcd14 <- "~/Documents/scRNAseq Neufeld Research/Winter Spring 2026/Real_Data_Simulations/pvals_countsplit_bcd14"
countsplit_files_bcd14 <- list.files(path = path_countsplit_bcd14, pattern = "\\Rds$", full.names = TRUE)
clean_names_countsplit_bcd14 <- tools::file_path_sans_ext(basename(countsplit_files_bcd14))
countsplit_pvals_bcd14 <- setNames(lapply(countsplit_files_bcd14, readRDS), clean_names_countsplit_bcd14)

# reorder because R orders 10 in front of 2
countsplit_pvals_bcd14 <- countsplit_pvals_bcd14[c(1,3,4,5,6,7,8,9,10,2)]

# Loading CADET null (nonzero off diagonals) p-values
path_cadet_null_bcd14 <- "~/Documents/scRNAseq Neufeld Research/Winter Spring 2026/Real_Data_Simulations/pvals_cadet_null_bcd14"
cadet_null_files_bcd14 <- list.files(path = path_cadet_null_bcd14, pattern = "\\Rds$", full.names = TRUE)
clean_names_cadet_null_bcd14 <- tools::file_path_sans_ext(basename(cadet_null_files_bcd14))
cadet_null_pvals_bcd14 <- setNames(lapply(cadet_null_files_bcd14, readRDS), clean_names_cadet_null_bcd14)

# reorder because R orders 10 in front of 2
cadet_null_pvals_bcd14 <- cadet_null_pvals_bcd14[c(1,3,4,5,6,7,8,9,10,2)]

# Loading CADET null indep p-values
path_cadet_indep_null_bcd14 <- "~/Documents/scRNAseq Neufeld Research/Winter Spring 2026/Real_Data_Simulations/pvals_cadet_indep_null_bcd14"
cadet_indep_null_files_bcd14 <- list.files(path = path_cadet_indep_null_bcd14, pattern = "\\Rds$", full.names = TRUE)
clean_names_cadet_indep_null_bcd14 <- tools::file_path_sans_ext(basename(cadet_indep_null_files_bcd14))
cadet_indep_null_pvals_bcd14 <- setNames(lapply(cadet_indep_null_files_bcd14, readRDS), clean_names_cadet_indep_null_bcd14)

# reorder because R orders 10 in front of 2
cadet_indep_null_pvals_bcd14 <- cadet_indep_null_pvals_bcd14[c(1,3,4,5,6,7,8,9,10,2)]

# Loading CADET demean (nonzero off diagonals) p-values
path_cadet_demean_bcd14 <- "~/Documents/scRNAseq Neufeld Research/Winter Spring 2026/Real_Data_Simulations/pvals_cadet_demean_bcd14"
cadet_demean_files_bcd14 <- list.files(path = path_cadet_demean_bcd14, pattern = "\\Rds$", full.names = TRUE)
clean_names_cadet_demean_bcd14 <- tools::file_path_sans_ext(basename(cadet_demean_files_bcd14))
cadet_demean_pvals_bcd14 <- setNames(lapply(cadet_demean_files_bcd14, readRDS), clean_names_cadet_demean_bcd14)

# reorder because R orders 10 in front of 2
cadet_demean_pvals_bcd14 <- cadet_demean_pvals_bcd14[c(1,3,4,5,6,7,8,9,10,2)]

# Loading CADET demean indep p-values
path_cadet_indep_demean_bcd14 <- "~/Documents/scRNAseq Neufeld Research/Winter Spring 2026/Real_Data_Simulations/pvals_cadet_indep_demean_bcd14"
cadet_indep_demean_files_bcd14 <- list.files(path = path_cadet_indep_demean_bcd14, pattern = "\\Rds$", full.names = TRUE)
```



```
clean_names_cadet_indep_demean_bcd14 <- tools::file_path_sans_ext(basename(cadet_indep_demean_files_bcd14))
cadet_indep_demean_pvals_bcd14 <- setNames(lapply(cadet_indep_demean_files_bcd14, readRDS), clean_names_cadet_indep_demean_bcd14)

# reorder because R orders 10 in front of 2
cadet_indep_demean_pvals_bcd14 <- cadet_indep_demean_pvals_bcd14[c(1,3,4,5,6,7,8,9,10,2)]
```

Assessing number of highly significant genes

```
source("find_sig_genes.R")

# Get number of significant genes (p < 0.001) for each method and each random seed
naive_bcd14_numsig <- find_sig_genes(naive_pvals_bcd14, p_val = 0.001)
countsplit_bcd14_numsig <- find_sig_genes(countsplit_pvals_bcd14, p_val = 0.001)
cadet_null_bcd14_numsig <- find_sig_genes(cadet_null_pvals_bcd14, p_val = 0.001)
cadet_indep_null_bcd14_numsig <- find_sig_genes(cadet_indep_null_pvals_bcd14, p_val = 0.001)
cadet_demean_bcd14_numsig <- find_sig_genes(cadet_demean_pvals_bcd14, p_val = 0.001)
cadet_indep_demean_bcd14_numsig <- find_sig_genes(cadet_indep_demean_pvals_bcd14, p_val = 0.001)

methods_tab <- data.frame(rbind(naive_bcd14_numsig, countsplit_bcd14_numsig, cadet_null_bcd14_numsig, cadet_indep_null_bcd14_numsig, cadet_demean_bcd14_numsig, cadet_indep_demean_bcd14_numsig),
  rownames(methods_tab) <- c("Naive", "PCS", "CADET (null)", "CADET (independent, null)", "CADET (demean)", "CADET (independent, demean)"),
  colnames(methods_tab) <- c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10))
methods_tab
```

##	1	2	3	4	5	6	7	8	9	10
## Naive	141	144	154	156	145	148	137	148	160	140
## PCS	99	99	111	111	97	101	104	102	115	110
## CADET (null)	0	2	19	78	0	0	0	0	0	138
## CADET (independent, null)	113	131	147	136	122	178	191	90	90	141
## CADET (demean)	7	76	62	104	97	2	3	2	3	117
## CADET (independent, demean)	117	135	147	136	122	182	198	100	92	141

CADET not consistent across random seeds (especially without independence assumption). CADET without independence does not identify significant genes (low power).

PCS seems to identify slightly fewer DE genes than naive

Pairwise correlations between methods

This will assess how stable different methods are when using the same random seed

```
# choose random seed 1 to demonstrate a case where CADET finds no significant genes
pvals_seed1 <- list(naive_pvals_bcells[[1]], countsplit_pvals_bcells[[1]],
  cadet_null_pvals_bcells[[1]], cadet_indep_null_pvals_bcells[[1]],
  cadet_demean_pvals_bcells[[1]], cadet_indep_demean_pvals_bcells[[1]])
pvals_seed1 <- pvals_seed1 %>% purrr::reduce(full_join, by = "Gene")
pvals_seed1 <- na.omit(pvals_seed1)
colnames(pvals_seed1) <- c("Gene", "naive", "countsplit", "cadet_null",
  "cadet_indep_null", "cadet_demean", "cadet_indep_demean")

seed1_cor_pearson <- cor(pvals_seed1[, -1], method = "pearson")
as.data.frame(seed1_cor_pearson)
```

##	naive	countsplit	cadet_null	cadet_indep_null
## naive	1.0000000	0.40962181	-0.37347830	0.23997651
## countsplit	0.4096218	1.00000000	-0.12871908	-0.07998401
## cadet_null	-0.3734783	-0.12871908	1.00000000	-0.05393395

```
## cadet_indep_null      0.2399765 -0.07998401 -0.05393395      1.00000000
## cadet_demean         -0.2734696 -0.24122977  0.59438255      0.22889850
## cadet_indep_demean   0.2511424 -0.06983103 -0.05949916      0.99952303
##
##          cadet_demean cadet_indep_demean
## naive          -0.2734696      0.25114240
## countsplit      -0.2412298     -0.06983103
## cadet_null       0.5943825     -0.05949916
## cadet_indep_null 0.2288985      0.99952303
## cadet_demean     1.0000000      0.22223233
## cadet_indep_demean 0.2222323      1.00000000
```

- Countsplit seems to have some correlation with naive, while the CADET results have much lower correlation
- CADET null and demean (both with independence) are highly correlated
- CADET null and demean (without independence) are also somewhat correlated, but less so than the two CADET methods with the independence assumption

```
seed1_cor_spearman <- cor(pvals_seed1[, -1], method = "spearman")
as.data.frame(seed1_cor_spearman)
```

```
##
##          naive countsplit cadet_null cadet_indep_null
## naive          1.00000000  0.6122461 -0.33271719      0.08294104
## countsplit      0.61224609  1.00000000 -0.20865195     -0.11734553
## cadet_null     -0.33271719 -0.2086519  1.00000000      0.05454112
## cadet_indep_null 0.08294104 -0.1173455  0.05454112      1.00000000
## cadet_demean   -0.25233188 -0.2224185  0.87002657      0.23291091
## cadet_indep_demean 0.09809051 -0.1033801  0.05034530      0.99943676
##
##          cadet_demean cadet_indep_demean
## naive          -0.2523319      0.09809051
## countsplit      -0.2224185     -0.10338006
## cadet_null       0.8700266      0.05034530
## cadet_indep_null 0.2329109      0.99943676
## cadet_demean     1.0000000      0.22826885
## cadet_indep_demean 0.2282688      1.00000000
```

- Countsplit and naive have some correlation
- CADET null and demean (both with independence) are highly correlated
- CADET null and demean (without independence) are also highly correlated, more so than the previous table using Pearson correlation